

Tecnológico de Monterrey

Evidencia2

Daniel Gómez Guerrero A01572201

18 de octubre del 2026

Estructura de Datos y Algoritmos Fundamentales (Gpo. 608)
David Alonso Cantú Delgado

Tecnológico de Monterrey

Evidencia de ejecución

Resultados del ordenamiento

Prompts utilizados

Durante la actividad, utilicé una cantidad moderada de inteligencia artificial, principalmente para asegurarme de que el código fuera estable y de que no siguieran apareciendo errores. Sin embargo, la ayuda de mi hermano fue vital para que lograra completar el código.

Aunque sí utilicé Copilot para corregir algunos aspectos del código, este no me ayudó a escribir tantas partes del código como tal; principalmente, lo utilicé para agregar comentarios en una gran parte del documento. No obstante, también me ayudó a implementar una sección de verificación con stoi para los años, cuyo significado no tenía claro hasta que mi hermano me ayudó a verificar que no fuera un error y a comprender qué hacía esa parte del código.

Por otro lado, sí utilicé un par de prompts en Gemini para apoyarme en la creación de mi código. Específicamente, le pregunté: «¿Por qué me está saliendo un error al generar mi archivo de salida, haciendo que solo aparezcan los mensajes de los logs?». Esto se debía a que no estaba muy seguro de dónde provenía la falla, ya que no era un error que se detectara en la terminal, pues el código sí se ejecutaba. Finalmente, Gemini me ayudó a darme cuenta de que el problema se debía a que una parte de mi código estaba sobrescribiendo parte de los logs, lo que provocaba que, al imprimirse, solo se tomara en cuenta la parte final, es decir, los mensajes.

Otro prompt que le hice a Gemini fue el siguiente: «¿Por qué me está fallando mi stringstream?». Le envié este mensaje junto con una imagen de la parte del código que estaba fallando, relacionada con el uso de stringstream para el día, el mes y el año que se debían buscar para encontrar todos los logs de una fecha determinada. Esto me ayudó a darme cuenta de que el error se debía a que estaba relacionando el proceso con tempLog.month, tempLog.day y tempLog.year, lo cual no era correcto porque esos elementos se utilizaban en otro stringstream diferente.

También le pedí a ChatGPT lo siguiente: «¿Puedes arreglar los errores gramaticales de lo siguiente?». Esto lo hice para asegurarme de que el texto escrito fuera del código estuviera correctamente redactado.

Al utilizar Gemini para investigar por qué algunas partes de mi código estaban fallando, recibí varias sugerencias que no eran viables. Por ejemplo, en una ocasión me recomendó utilizar un método llamado getSize() de mi clase Queue, aunque mi clase no contaba con dicho método. Esto se debía a que Gemini no conocía mi código ni mis clases por completo, sino únicamente algunas secciones de estos. Por ello, en varias ocasiones tuve que ignorar sus sugerencias o explicarle directamente que no podía implementarlas porque no contaba con los métodos o elementos necesarios para que fueran útiles.

Tecnológico de Monterrey

Reflexión personal

En esta evidencia, mi uso de la IA fue muy útil para comprobar dónde o en qué sección estaban apareciendo errores y por qué, a la vez que agilizó el proceso de agregar comentarios gracias a Copilot. Normalmente, diría que la falta de su uso no habría cambiado mucho, pero eso no es necesariamente cierto. Aunque una gran parte de mi código fue hecha reutilizando y adaptando códigos anteriores de actividades previas (incluyendo la evidencia anterior), no estoy seguro de si habría tenido el tiempo suficiente para terminar y comprobar que el código estuviera completamente funcional si la IA no me hubiera ayudado a detectar el origen de los errores y a proporcionar sugerencias.

No tengo duda de que, con la ayuda de mi hermano y mediante la investigación, podría haber logrado eventualmente que el código alcanzara la misma calidad (o, por lo menos, una calidad similar) a la que tiene actualmente. Sin embargo, habría tardado significativamente más en encontrar el origen de los errores y, por ende, sus soluciones, lo cual posiblemente me habría llevado a entregar la actividad tarde.

Ahora, especialmente en la Parte B de la actividad, se implementaron algunas de las estructuras vistas en clase; específicamente, Stack, Queue y DoublyLinkedList. Se utilizó Queue para poder leer y encolar cada log de los archivos provistos, siendo esencial en la parte de lectura de datos de mi código, aunque tal vez sería posible hacerlo con DoublyLinkedList, a costa de necesitar más líneas de código.

Por otro lado, en la opción 3 del menú de usuario, se utilizó Stack para invertir el orden de los registros provistos en la opción 2 o de los registros correspondientes a una fecha específica definida por el usuario, en caso de que se hubiera consultado una fecha previamente. Teóricamente, hubiera sido posible hacer lo mismo con DoublyLinkedList debido a su capacidad de recorrer los nodos de una lista hacia adelante y hacia atrás. Sin embargo, esto habría requerido una mayor cantidad de líneas de código, ya que sería más complicado de implementar que una pila. Además, una pila, al ser una estructura LIFO, era una forma segura de invertir el orden de los valores dados, mientras que DoublyLinkedList tendría una mayor probabilidad de provocar errores durante su implementación, lo que implicaría dedicar más tiempo a la programación y tener más cuidado. Sin contar que es más sencillo entender el funcionamiento de un Stack que el de una DoublyLinkedList, aunque esta última sería un poco más eficiente en cuanto al uso de memoria, ya que no requeriría una lista auxiliar para realizar la operación.